

Lightweight Federated Learning for On-Device Non-Intrusive Load Monitoring

Yehui Li¹, Graduate Student Member, IEEE, Ruiyang Yao, Graduate Student Member, IEEE,
Dalin Qin¹, Graduate Student Member, IEEE, and Yi Wang¹, Senior Member, IEEE

Abstract—Non-intrusive load monitoring (NILM) is a critical technology for disaggregating appliance-specific energy usage by only observing household-level power consumption. If NILM can be performed on end devices (such as smart meters), it can facilitate electricity demand identification and electricity behavior perception for real-time demand-side energy management. However, implementing high-performance NILM models on end devices presents an unresolved issue, encompassing two primary challenges: hardware resource constraints and data resource paucity on end devices. To this end, this paper proposes a lightweight federated learning approach for on-device NILM by combining neural architecture search (NAS) and federated learning. Firstly, a memory-efficient NAS approach is investigated to determine a personalized model within the resource constraints of end devices. Secondly, a federated mutual learning approach is designed to orchestrate the cooperation of distributed end devices with heterogeneous personalized models in a privacy-preserving manner. Case studies on two real-world datasets verify that the proposed method for appliance-level power disaggregation outperforms conventional methods in accuracy and efficiency.

Index Terms—Non-intrusive load monitoring, on-device training, neural architecture search, federated learning.

I. INTRODUCTION

ACCORDING to the International Energy Agency (IEA) report, residential and commercial buildings account for 26% of the global energy sector [1], and energy conservation at building levels is imperative. Non-intrusive load monitoring (NILM), also known as load disaggregation, is the key to a better understanding of buildings' energy consumption by identifying appliance-level loads [2]. By providing detailed insights into energy consumption patterns, NILM empowers consumers to make informed decisions and adopt energy-saving habits, thus improving energy efficiency [3]. Furthermore, NILM can be employed for appliance-level monitoring and fault detection, thereby facilitating the maintenance

and repair of electrical systems [4]. To achieve real-time and personalized energy monitoring, the disaggregation models can be developed on end meters to locally analyze the collected high-frequency power data [5]. However, the application of on-device NILM is a challenging and under-researched topic, which faces two main challenges: 1) constrained resources on the end device can not support the training of complicated models due to memory overruns, and 2) insufficient data collected by a single end device may lead to poor model generalization due to overfitting.

Machine learning models, particularly deep learning methods, have been extensively applied to NILM problems for analyzing power consumption data and accurately disaggregating electricity usage into individual appliances. Reference [6] proposed a sequence-to-sequence long short-term memory (LSTM) model for NILM, which utilized the Hilbert transform for feature formulation. Reference [7] presented a sequence to point convolutional neural networks (CNN) for effective energy disaggregation. Reference [8] applied the double Fourier integral analysis for feature extraction, then input the feature to a CNN network for accurate load disaggregation. In addition, [9] proposed a two-stream convolutional neural network (TSCNN) to extract load features from the temporal and spectral load signatures for appliance recognition. Reference [10] introduced a self-attention mechanism into the temporal convolutional network (TCN) for state detection and energy disaggregation. Moreover, [11] proposed a fully convolutional denoising auto-encoder architecture tailored for large non-residential buildings. Reference [12] applied a middle window transformer model to NILM with transferability enhancement. Reference [13] proposed two transfer learning schemes for the transition of the model between different appliances. Reference [14] identified the need to model unknown appliances in NILM and formulated a generative network utilizing variational autoencoders and capsule networks for accurate disaggregation.

Typically, the computation- and memory-intensive overhead required for training neural network-based models can be prohibitive for resource-constrained end devices such as smart meters. Recently, several works have focused on how to deploy NILM models on edge devices, such as smart meters. Reference [15] provided a potential solution for an edge NILM system by implementing a convolutional neural network on an Arm Cortex-M7 microcontroller unit (MCU). References [16] and [17] presented a feature extraction approach to lower the feature dimension for NILM to reduce the computation and

Received 26 February 2024; revised 3 July 2024 and 5 September 2024; accepted 14 October 2024. Date of publication 17 October 2024; date of current version 21 February 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFE0141200, and in part by the Research Grants Council of the Hong Kong SAR under Grant HKU 27203723. Paper no. TSG-00316-2024. (Corresponding author: Yi Wang.)

The authors are with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, SAR, China, and also with the Shenzhen Institute of Research and Innovation, The University of Hong Kong (Shenzhen), Shenzhen 518057, China (e-mail: yhli@eee.hku.hk; ryyao@connect.hku.hk; dlqin@eee.hku.hk; yiwang@eee.hku.hk).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TSG.2024.3482363>.

Digital Object Identifier 10.1109/TSG.2024.3482363

storage overhead on MCU devices. Reference [18] employed a commercial tool, Tensorflow Lite, to quantize and compress the CNN model implemented on an Android platform. Reference [19] adopted a performance-aware model pruning strategy to decrease computational complexity. Reference [20] proposed a lightweight solution for edge NILM by combining metric-based meta-learning and supervised pre-training. The above studies only provide scenarios for model deployment and ignore more resource-demanding model training. One related work is [21], which provides an efficient pruning solution applied before full training. Another investigation [22] combined model pruning and unsupervised transfer learning to reduce memory consumption for edge NILM. Nevertheless, all of the aforementioned research has considered resource overhead reduction by only compressing fixed architectural models rather than searching for the optimal architecture in a compressed space. The pre-designed NILM models with fixed architecture may not be suitable for all appliances and households. Recently, an automated architecture design technique called neural architecture search (NAS) [23] has been developed, which can discover high-performance networks on specific tasks. Reference [24] improved the efficiency of NAS by forcing the sharing of parameters across child models during the search for architecture. By constructing a differential search space, [25] presented an efficient gradient-based architecture search algorithm that applies to convolutional and recurrent architectures. On the basis, [26] provided a path-level pruning method using binarization to directly search architecture on the target task. However, NAS inherently requires tremendous computation resources, hindering the end device's adoption.

Another limitation of most existing on-device NILM methods is their reliance on local data exclusively for model training, thereby neglecting the potential enhancement of prediction accuracy by incorporating multiple data resources. However, directly sharing high-frequency smart meter data that may contain private information is subjected to strict regulations [27]. To improve model performance by utilizing distributed data resources, federated learning (FL) [28] is a promising solution as it enables the distributed collaboration of multiple devices without revealing their local raw data. Several FL-based methods have been carried out for on-device NILM. Reference [29] proposed a sequence-to-point federated learning framework to co-model the NILM problem among several distributed parties. Besides, [30] investigated blockchain-assisted federated learning approach to enhance data safety. Reference [31] combined differential privacy and federated learning to make a trade-off between data utility and consumer privacy. Reference [32] integrated transfer learning into federated learning to improve model transferability across different devices. The aforementioned FL-based methods can only be applied for the collaborative training of models with identical architecture.

To bridge the above research gaps, this paper proposed a lightweight federated learning approach to decompose the main power into appliance-level power in real-time on resource-constrained end devices, making the following contributions:

- Propose a memory-efficient NAS method to construct high-performance personalized models tailored to individual appliances and households on end devices. The integration of compressed search space, single-path search strategy, and hardware-aware evaluation can achieve a substantial reduction in computational resources.
- Develop an adaptive federated mutual learning approach to train personalized models with distributed data on end devices in a privacy-preserving manner. The introduced knowledge distillation mechanism facilitates mutual enhancement between global proxy models and local personalized models through knowledge transfer.
- Conduct extensive case studies on two real-world open datasets with various appliances of different households. The experimental results and complexity analysis demonstrate the superior accuracy and efficiency of the proposed method for on-device NILM.

The remainder of this paper is organized as follows: Section II defines and formulates the on-device NILM problem to be solved. Section III provides the framework and details the algorithms for the power disaggregation task. Section IV conducts case studies and makes comparisons to validate the effectiveness of the lightweight federated learning approach. Section V draws the conclusions.

II. PROBLEM STATEMENT

The NILM can be formulated as the problem of estimating appliance-level power consumption with only aggregated power consumption. Given the main power P_t measured by bulk meter, the objective is to accurately disaggregate the real power $[P_{1,t}, \dots, P_{N,t}]$ for N appliances behind the meter over a specific time window [2]. The aggregated power at time t can be expressed as follows:

$$P_t = \sum_{n=1}^N P_{n,t} + X_t + e_t \quad (1)$$

where X_t denotes the steady power consumption of the unknown appliances and e_t denotes the residual noise. To generate estimates $[\hat{P}_{1,t}, \dots, \hat{P}_{N,t}]$ with minimal error from the real power, for each appliance n , we need to train an estimation model $f_n(\cdot)$ with weight parameters w_n on the end device to fit the mapping relationship between the sequences. Formally, the overall estimation process for n -th appliance is defined as:

$$\hat{P}_{n,t} = f_n(w_n, P_t) \quad (2)$$

The choice of the on-device NILM model generally encounters a dilemma: due to memory constraints, implementing a large-scale model with strong generalization ability on the end device is challenging; however, a small model with a fixed architecture may struggle to perform satisfactorily on all decomposition tasks for appliances that exhibit remarkably distinct consumption patterns. NAS, an automated method for discovering the best neural network architecture for a specific task, can be used to solve this dilemma [23].

For NAS, the gradient-based approach is more suitable for on-device NILM due to its high computational efficiency [25].

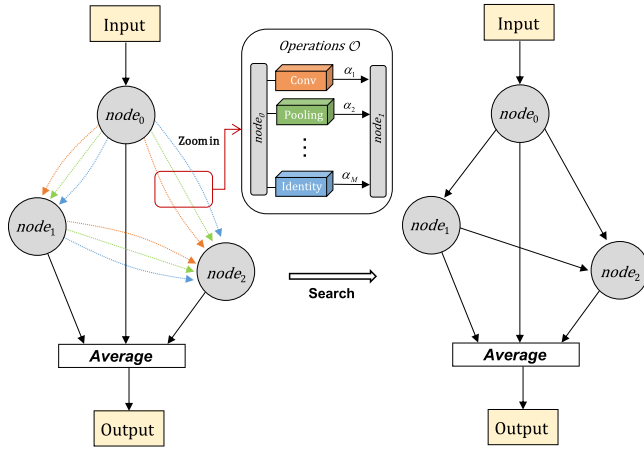


Fig. 1. Illustration of NAS process.

Hence, we adopt this NAS technique to search for an on-device personalized model for the target appliance that can more effectively adapt to local consumption behavior. NAS aims to search for the optimal architecture on a supernet. As shown in Fig. 1, a supernet can be considered a directed acyclic graph composed of nodes (which denote neurons of the neural networks) and edges (which denote operations between neurons). Let $\mathcal{O}$ be the set of candidate operations (e.g., convolution, pooling, zero, identity, etc.) for each directed edge. The architecture parameters α_n represent the importance of the candidate operations, where high importance means the corresponding operation has a stronger learning ability for the whole task. Consequently, NAS can search for the optimal operation by optimizing α_n . The objective of NAS can be formulated as a bi-level optimization problem:

$$\begin{aligned} \min_{\alpha_n} \quad & \mathcal{L}_{\text{val}}(w_n^*(\alpha_n), \alpha_n) \\ \text{s.t.} \quad & w_n^*(\alpha_n) = \operatorname{argmin}_{w_n} \mathcal{L}_{\text{train}}(w_n, \alpha_n) \end{aligned} \quad (3)$$

where $\mathcal{L}_{\text{val}}$ and $\mathcal{L}_{\text{train}}$ are validation loss and training loss respectively. Typically, we first solve the constraints and then minimize the objective in (3). To avoid complex inner optimization, the approximation of the upper-level problem can be rewritten as:

$$\begin{aligned} \nabla_{\alpha_n} \mathcal{L}_{\text{val}}(w_n^*(\alpha_n), \alpha_n) \\ \approx \nabla_{\alpha_n} \mathcal{L}_{\text{val}}(w_n - \xi \nabla_{w_n} \mathcal{L}_{\text{train}}(w_n, \alpha_n), \alpha_n) \end{aligned} \quad (4)$$

where ξ is the learning rate, and the upper-level variable w_n^* is approximated by the value w_n of a single-step update instead. Consequently, we can solve the objective in (3) by employing an alternative gradient descent strategy. Finally, the operations connected to each node with the maximum architecture parameters are reserved to construct the personalized model. Note that the number of weight parameters w_n for a supernet consisting of multiple candidate networks is quite large. Therefore, caching w_n along with the gradients and intermediate activations during the search process can be challenging for end devices with small memory capacity.

The elaborately developed model on the end device is susceptible to the risk of overfitting during the training process due to limited data availability. Moreover, privacy concerns

impede the sharing of high-frequency power consumption data containing sensitive consumer information among various end devices. To address the privacy issue, we utilize a federated learning technique to enhance personalized models in capturing global consumption patterns in a privacy-preserving manner [33]. Suppose that there are K households participating in the collaboration, each end device updates the model with locally collected data x_n^k and then uploads model weights w_n^k to the server. The objective of vanilla federated learning, i.e., FedAvg, can be formulated as:

$$\min_{w_n} \mathcal{L}(\bar{w}_n) = \frac{1}{K} \sum_{k=1}^K \mathcal{L}_{\text{train}}(w_n^k, x_n^k) \quad (5)$$

where $\mathcal{L}$ denotes the global loss, and $\bar{w}_n$ denotes the average weight obtained by aggregating the local weight parameters w_n^k of K on-device models on the server. For simplicity, the weight parameters, architecture parameters, and data for n -th appliance of k -th household are represented as w , α , and x , respectively, in the following sections.

The integration of NAS and federated learning for on-device NILM should address two main issues within the proposed decentralized framework:

- 1) The storage and updating of the supernet can be resource-intensive, potentially leading to memory explosion on end devices. The first challenge is to develop a memory-wise strategy to reduce the memory footprint of NAS to the same level as regular training.
- 2) The personalized models customized for different households typically have different network architectures. The second challenge is to establish a flexible mechanism to aggregate the weight parameters from heterogeneous models in federated learning.

III. METHODOLOGY

In this section, we present the overall framework and the implementation details of the proposed on-device method to tackle the abovementioned problems.

A. Framework

The proposed lightweight federated NILM method aims to develop high-performance models on end devices for accurately estimating appliance-level power consumption. Given the constrained memory and insufficient data available on the end device, we propose a novel framework depicted in Fig. 2, which consists of four steps:

- 1) *Search*: The personalized model is automatically searched from the supernet using the NAS technique according to the characteristics of local data measured by the end device. The designed model architecture varies depending on the appliances and households involved.
- 2) *Mutual Distillation*: The unified proxy model with the same network architecture is introduced for each end device. Both the proxy model and the personalized model are trained with collected data. Meanwhile, the proxy model can transfer global knowledge to and

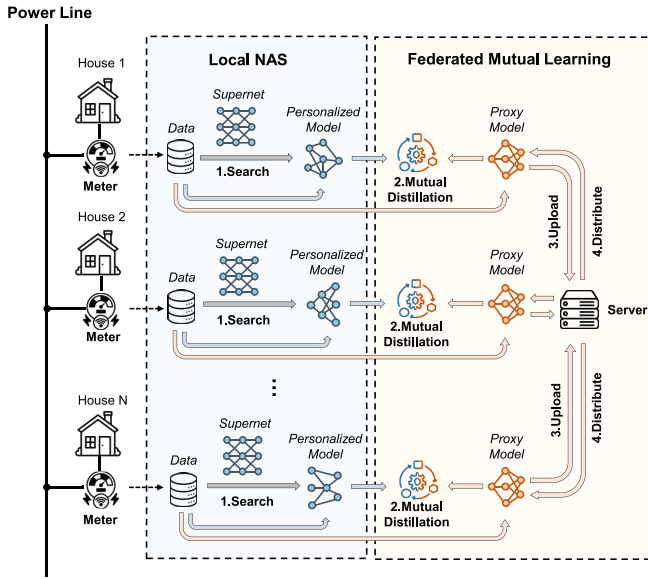


Fig. 2. Overview of the proposed framework for NILM.

obtain local knowledge from the specialized personalized model via mutual distillation.

- 3) *Upload*: The weight parameters of the proxy models on all end devices are uploaded to the server for average aggregation. Note that the raw load data and the parameters of the personalized model are retained on the end device and not exposed to other entities.
- 4) *Distribute*: The aggregated weight parameters are distributed to each end device to update the proxy model. The iterative process from Steps 2) to 4) is then repeated for federated training. Finally, each end device utilizes its data to fine-tune the personalized model to match local data distribution.

B. Memory-Efficient Neural Architecture Search

As an optimization problem, NAS generally involves designing a search space, i.e., a supernet, as the feasible domain, followed by utilizing a sophisticated search strategy to find the optimal architecture and ultimately evaluating the performance of search results. The weight parameters for the operations in the search space and the intermediate parameters (activations, gradients) generated in architecture search may cause considerable memory overhead. To make this process viable on resource-constrained end devices, we develop a memory-efficient NAS (MNAS) algorithm shown in Fig. 3, including the compressed search space, the single-path search strategy, and the hardware-aware evaluation.

1) *Compressed Search Space*: CNN networks have gained popularity in the field of NILM due to their ability to effectively learn complex patterns and features from time-series data [34]. The candidate operations will constitute a combinatorially large search space. Note that the parameterized convolutional operations occupy a significant portion of the storage space compared to other non-parameterized operations. To reduce the memory footprint, we compress all candidate convolutional operations per layer into a single net

by sharing the kernel weights. Specifically, the weights of the small kernels can be viewed as the subnet of the large kernels. Let $w_{1 \times 1}$, $w_{3 \times 3}$ and $w_{5 \times 5}$ denote the 1×1 , 3×3 and 5×5 kernel for an convolutional layer respectively. Hence, the compressed search space can be formulated as:

$$w_{5 \times 5} = w_{1 \times 1} + w_{3 \times 3|1 \times 1} + w_{5 \times 5|3 \times 3} \quad (6)$$

where $w_{3 \times 3|1 \times 1}$ and $w_{5 \times 5|3 \times 3}$ denote the 3×3 and 5×5 convolutional kernels excluding subsets, respectively. Here the 3×3 kernel can be equivalent to the combination of $w_{1 \times 1}$ and $w_{3 \times 3|1 \times 1}$. As shown in Fig. 3, each end device only needs to store the largest kernel rather than the entire supernet, to include all subsets of convolutional kernels. In contrast to traditional NAS, the memory consumption of compressed search space does not grow linearly with the number of candidate convolutional operations per layer.

2) *Single-Path Search Strategy*: The differential NAS employs standard gradient descent to jointly update the weight parameters w and architecture parameters α . However, the architecture search of the supernet is conducted for all paths (i.e., the exponential number of operations), which may give rise to the memory overflow issue. To decrease the memory consumption of architecture search to a level comparable to compact model training, we introduce path binarization, where only one path is selected for each round of updates.

For a specific edge with parallel paths, the node output m_O is a mixed operation based on the outputs $[o_1, \dots, o_M]$ of M paths [26]. Given input x , the mixture value $m_O(x)$ can be calculated as:

$$m_O(x) = \sum_{i=1}^M \frac{\exp(\alpha_i)}{\sum_{j \in O} \exp(\alpha_j)} o_i(x) \quad (7)$$

where architecture parameter α_i is weighted by applying the softmax function. As shown in (7), the end device is required to cache intermediate activations and gradients for the operations of all M paths. In contrast, training the searched models involves only one path. Therefore, we adopt the single-path strategy that freezes other paths when updating the supernet parameters to reduce memory footprint. Specifically, the paths are discretized and sampled by the binary gates G as:

$$g = G(p_1, \dots, p_M) = \begin{cases} [1, 0, \dots, 0] & \text{with prob. } p_1 \\ \dots & \\ [0, 0, \dots, 1] & \text{with prob. } p_M \end{cases} \quad (8)$$

where g denotes a binary vector and the value of variable g_i indicates whether or not the i -th path is sampled. p denotes the sampling probability obtained through the softmax transformation of α . Based on the binary gates, the output of the mixed operation can be expressed as:

$$m_O^{\text{binary}} = \sum_{i=1}^M g_i o_i(x) = \begin{cases} o_1(x) & \text{with prob. } p_1 \\ \dots & \\ o_M(x) & \text{with prob. } p_M \end{cases} \quad (9)$$

Fig. 3 illustrates the training process of the binarized architecture search in the supernet. We can observe that the discrete selection ensures that only one executed path is activated

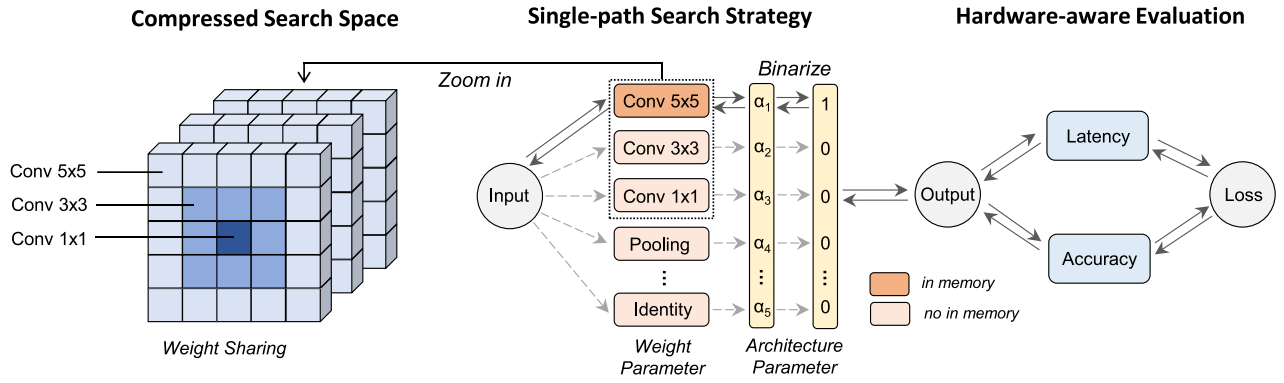


Fig. 3. The proposed memory-efficient NAS with compressed search space, single-path search strategy, and hardware-aware evaluation.

in memory, thus preventing all candidate operations from occupying memory at the same time.

However, the incorporation of binary gates results in that losses are not directly differentiable to the sampled architecture parameter α_i during gradient descent. To circumvent this issue, we employ the Gumbel-Softmax function [35] to relax the discrete binary variables to be continuous during backpropagation. Specifically, given the probability vector $[p_1, \dots, p_M]$, we sample paths into binary vector g using random Gumbel noise:

$$g = \operatorname{argmax}_i \{\log p_i + \pi_i\} \quad (10)$$

where $\pi_i = -\log(-\log(u_i))$ follows the Gumbel distribution and $u_i \sim \text{Uniform}(0, 1)$. Considering the non-differentiable property of argmax , we further replace the argmax operation with the continuous softmax operation as:

$$g_i^{\text{cont}} = \frac{\exp((\log p_i + \pi_i)/\tau)}{\sum_{i=1}^M \exp((\log p_i + \pi_i)/\tau)} \quad (11)$$

where τ is the temperature parameter introduced to control the approximate discretization of the softmax function. As the temperature τ decreases, the vector g^{cont} converges to the one-hot binary vector g in (8). In this case, we can approximately calculate the gradient of architecture parameter α_i as:

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{val}}}{\partial \alpha_i} &= \sum_{j=1}^M \sum_{k=1}^M \frac{\partial \mathcal{L}}{\partial g_k} \cdot \frac{\partial g_k}{\partial p_j} \cdot \frac{\partial p_j}{\partial \alpha_i} \\ &\approx \sum_{j=1}^M \sum_{k=1}^M \frac{\partial \mathcal{L}_{\text{val}}}{\partial g_k} \cdot \frac{\partial g_k^{\text{cont}}}{\partial p_j} \cdot \frac{\partial p_j}{\partial \alpha_i} \\ &= \sum_{j=1}^M \sum_{k=1}^M \frac{\partial \mathcal{L}_{\text{val}}}{\partial g_k} \cdot p_j (\delta_{j,k} - g_j g_k) (\theta_{i,j} - p_i) \end{aligned} \quad (12)$$

where the indicator variables $\delta_{j,k}$ and $\theta_{i,j}$ can be expressed as:

$$\delta_{j,k} = \begin{cases} \frac{1}{\tau} & \text{if } j = k \\ 0 & \text{if } j \neq k \end{cases} \quad \theta_{i,j} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad (13)$$

It is clear in (12) that the differentiability issue of architecture parameters in single-path search is addressed by using the Gumbel-Softmax function. Regardless of the value of M , only one path is involved in each search step, significantly saving the end device's memory consumption.

3) *Hardware-Aware Evaluation*: Real-time NILM imposes a time constraint on model inference latency to enable continuous monitoring. Hence, we attempt to achieve a trade-off between accuracy and latency when searching for the architecture of the personalized model. Let $L(o_i)$ denote the measured hardware latency of i -th operation. To make latency differentiable to the architecture parameters, we model the final network latency T as the total parameter-weighted latency of the mixed operation for E edges as follows:

$$T(\alpha) = \sum_{e=1}^E \sum_{i=1}^M g_{e,i}(\alpha) L(o_i) \quad (14)$$

where $g_{e,i}$ denotes the binary variable of the i -th operation on the e -th edge. The intuition is that more complex models generally exhibit stronger representation ability but lead to longer inference times. In particular, the NILM problem requires only that the latency does not exceed the disaggregated window length. To search for the optimal architecture within the latency constraint, we incorporate the latency into the final loss function. The objective of hardware-aware architecture evaluation can be expressed as:

$$\begin{aligned} \min_{\alpha} \quad & \mathcal{L}_{\text{val}}(w, \alpha) + \left(\frac{T(\alpha)}{T_0} \right)^{\lambda} \\ \text{s.t.} \quad & w = \operatorname{argmin}_w \mathcal{L}_{\text{train}}(w, \alpha) \end{aligned} \quad (15)$$

where T_0 denotes the time length of the sliding window used in NILM and λ denotes a penalty factor greater than 1. With an appropriate λ , the second term can ensure that the latency penalty is activated when $T > T_0$ and hardly affects the loss value when $T < T_0$.

To search for the architecture parameter α_i , gradient descent can be used to solve the optimization problem (15). Thus, the key is to calculate the gradient. The gradient of the first term in (15) over α_i has been provided in (12). However, the gradient of the second term, denoted as LAT, is hard to compute. Nevertheless, we can approximately get the gradient of LAT over α_i as:

$$\frac{\partial \text{LAT}}{\partial \alpha_i} = \sum_{j=1}^M \sum_{k=1}^M \frac{\partial \text{LAT}}{\partial g_k} \cdot \frac{\partial g_k}{\partial p_j} \cdot \frac{\partial p_j}{\partial \alpha_i}$$

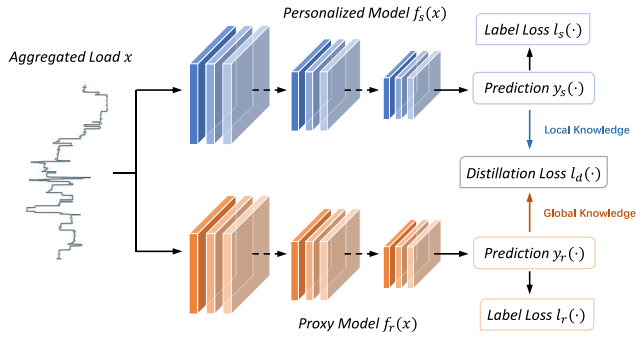


Fig. 4. Schematic of mutual learning with knowledge distillation.

$$\begin{aligned}
 & \approx \sum_{j=1}^M \sum_{k=1}^M \frac{\partial \text{LAT}}{\partial g_k} \cdot \frac{\partial g_k^{\text{cont}}}{\partial p_j} \cdot \frac{\partial p_j}{\partial \alpha_i} \\
 & = \frac{\lambda T^{\lambda-1}}{T_0^\lambda} \sum_{j=1}^M \sum_{k=1}^M \frac{\partial T}{\partial g_k} \cdot p_j (\delta_{j,k} - g_j g_k) (\theta_{i,j} - p_i) \quad (16)
 \end{aligned}$$

Benefiting from the approximation scheme, the above complex optimization problem can be solved through gradient descent efficiently. Ultimately, an optimal solution that strikes a balance between accuracy and latency can be reached.

C. Adaptive Federated Mutual Learning

Typically, federated learning can be regarded as a method to exchange knowledge between distributed devices by aggregating model weight. To cope with the heterogeneity of personalized models, we adopt mutual distillation for federated learning by introducing a unified proxy model. As shown in Fig. 4, each end device updates the personalized model and proxy model based on both label loss of local data, i.e., ℓ_s and ℓ_r , and mutual distillation loss ℓ_d between the two models. Specifically, the beneficial information is bidirectionally transferred through knowledge distillation. Since the personalized model has tailored architecture to local data distribution, the deeply drawn local knowledge can guide the proxy model training. Furthermore, since the proxy models are collaboratively aggregated among various devices, the personalized model can benefit from the global knowledge of the proxy model. Hence the final loss functions of the personalized model and proxy model can be formulated as:

$$\begin{aligned}
 \mathcal{L}_{\text{train},s} &= \ell_s(w_s, x) + \ell_d(w_s, w_r, x) \\
 \mathcal{L}_{\text{train},r} &= \ell_r(w_r, x) + \ell_d(w_s, w_r, x) \quad (17)
 \end{aligned}$$

The mutual knowledge distillation mechanism drives both models to converge in the same optimal direction. However, during the initial training, unreliable predictions may lead to the sharing of detrimental knowledge between the two models. Therefore, we adopt an adaptive approach to adjust the weight of the distillation loss in accordance with the convergence process. Let $y_s = f_s(w_s, x)$ and $y_r = f_r(w_r, x)$ denote the predictions of the personalized model and the proxy model,

Algorithm 1: Lightweight Federated Learning

```

function On-device NILMK,  $\mathcal{O}$ ,  $R$ 
  for each household  $k$  in  $K$  do
    Construct compressed search space  $w_{5 \times 5}$ ;
    Generate  $o_i$  parameterized by  $\alpha_i$  for each path
    for each operation  $i \in \mathcal{O}$  do
      Calculate single-path mixture  $m_{\mathcal{O}}$ 
      Compute gradients  $\nabla_w$  and  $\nabla_\alpha \leftarrow \frac{\partial \mathcal{L}_{\text{val}}}{\partial \alpha_i} + \frac{\partial \text{LAT}}{\partial \alpha_i}$ 
      Update model parameters:
       $\alpha \leftarrow \alpha - \xi_\alpha \nabla_\alpha$   $w \leftarrow w - \xi_w \nabla_w$ 
    return personalized model with weight  $w_s$ 
  for each round  $t$  in  $R$  do
    for each household  $k$  in  $K$  do
      Compute gradients  $\nabla_{w_s} \mathcal{L}_{\text{train},s}$  and  $\nabla_{w_r} \mathcal{L}_{\text{train},r}$ 
      Update personalized model and proxy model:
       $w_s \leftarrow w_s - \xi_{w_s} \nabla_{w_s}$   $w_r \leftarrow w_r - \xi_{w_r} \nabla_{w_r}$ 
      Upload proxy model weight  $w_r$ 
      Distribute aggregated proxy model weight:
       $\bar{w}_r \leftarrow \frac{1}{K} \sum_{k \in K} w_r^k$ 
    return global proxy model with weight  $\bar{w}_r$ 
  
```

respectively. Given the dataset containing main power x and appliance power y , the label losses can be expressed as:

$$\begin{aligned}
 \ell_s(w_s, x) &= \ell(y, y_s) \\
 \ell_r(w_r, x) &= \ell(y, y_r) \quad (18)
 \end{aligned}$$

where ℓ is a basic loss function which can be L2 loss. We can formulate the adaptive distillation loss as follows:

$$\ell_d(w_s, w_r, x) = \frac{\ell(y_s, y_r)}{\ell_s(w_s, x) + \ell_r(w_r, x)} \quad (19)$$

Here the distillation loss term will be weakened when the two models achieve poor performance. The full algorithm for the proposed method is summarized in Algorithm 1.

IV. CASE STUDIES

A. Experimental Setups

1) *Dataset Description*: To validate our proposed method, we test our model on two publicly available datasets, REFIT [36] and REDD [37]. REFIT contains appliance-level power consumption of 20 houses in the U.K., where every house is installed with 10 power sensors comprising a current clamp for the household aggregate and 9 individual appliance monitors (IAMS) with a time resolution of 8 seconds. REDD consists of whole-home and appliance-specific electricity consumption for 6 real houses in the USA with a time resolution of 3 seconds, where each of these houses contains 9 to 24 different appliances. Our case study follows the setup of previous research [29], where a small amount of pre-collected labeled appliance data is used for model training.

2) *Model Settings*: We use 5 benchmarks to illustrate the strength of our proposed models. The centralized model is trained using all data from different households by assuming no privacy concerns. The local model is trained solely using

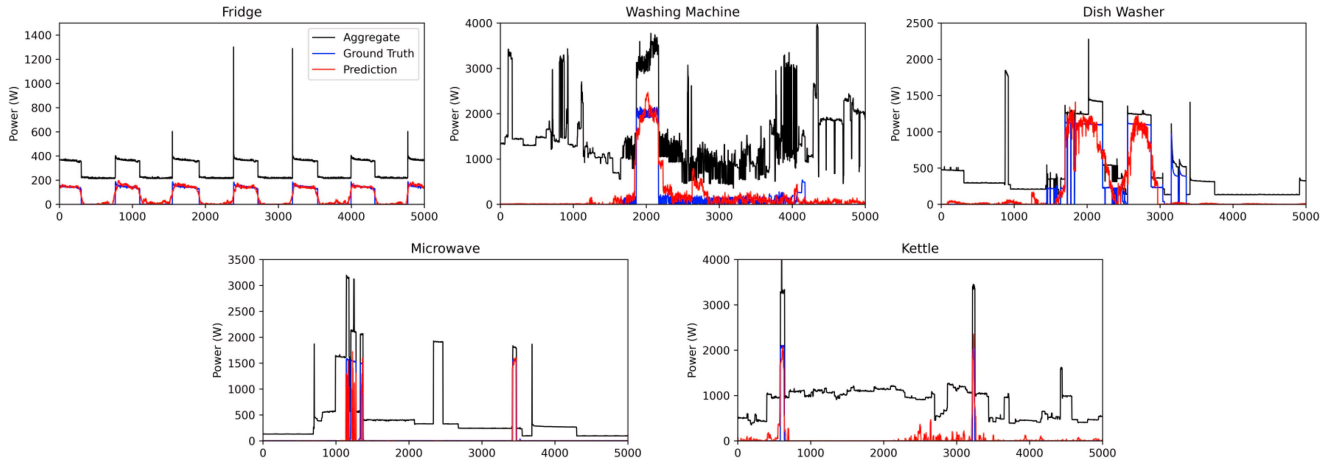


Fig. 5. Load disaggregation results for fridge, washing machine, dish washer, microwave, and kettle.

data collected by the end device for a single household. NAS and MNAS perform an architecture search before full model training on single household data. The federated model employs FedAvg [38] across households.

Our framework is a basic one that can be compatible with any state-of-the-art model and network used in general NILM tasks. In our experiments, we choose one-dimensional CNN, which is of interest in NILM, as the backbone. The fixed architecture for non-search models involves five convolutional layers with a 7-size kernel, followed by one dense layer. The superiority of the search space can be assessed by space diversity, efficiency evaluation, and comparative analysis. To find the optimal architecture, we carefully design a search space, which consists of 5 nodes and contains 8 candidate operations for each path, including {conv_7, conv_5, conv_3, conv_1, max pooling, average pooling, skip and identity}. The developed search space with enriched architecture types has been validated to yield device-friendly memory overhead and achieve significant performance on all appliances and households. After the search phase, the first two paths with the largest architecture parameters are retained for each node. Taking into account the trade-off between model communication and model representation capacity, the proxy models are composed of three convolutional layers with a kernel size of 5 and a single dense layer. The code for the experiments has been publicly available.¹

3) *Evaluation Metrics:* We use two metrics to measure the prediction accuracy for a single appliance: mean absolute error (MAE) and signal absolute error (SAE). MAE measures the average deviation between the estimates $\hat{y}_t$ and true values y_t , and SAE measures the deviation between the estimated total energy consumption and true consumption. The expressions are as follows:

$$\text{MAE} = \frac{1}{T} \sum_{t=1}^T |y_t - \hat{y}_t| \quad (20)$$

$$\text{SAE} = \frac{\left| \sum_{t=1}^T \hat{y}_t - \sum_{t=1}^T y_t \right|}{\sum_{t=1}^T y_t} \quad (21)$$

¹<https://github.com/hkuedl/Lightweight-Federated-NILM>

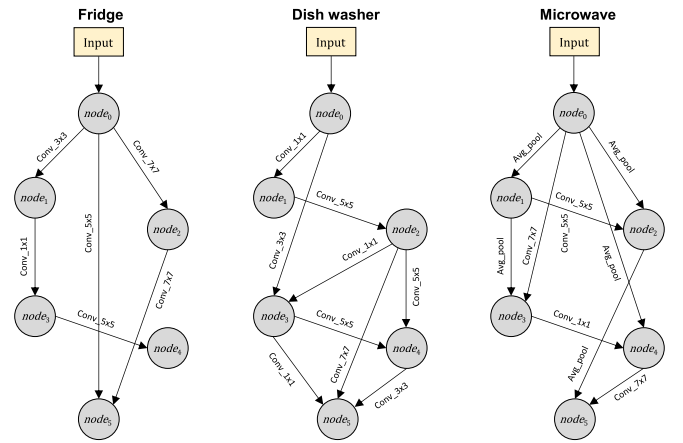


Fig. 6. The searched network architecture for different appliances with distinct consumption patterns.

B. Basic Results

Fig. 5 displays the predicted consumption for five different appliances, with distinct patterns observed. The power consumption of fridges exhibits a periodic pattern as they operate on a cooling cycle, where the compressor turns on periodically to cool the interior. Washing machines and dish washers are typically used continuously over a certain period, leading to sustained energy consumption. Conversely, consumption patterns for appliances like microwaves and kettles are typically abrupt and short-lived. Our proposed model captures the periodic usage pattern with high precision. Our model predicts consumption with minor deviations for aperiodic usage patterns with varying time durations. The different usage patterns are also reflected in the searched architectures, as shown in Fig. 6. For the fridge, the architecture of NAS includes few convolutional layers because cyclic power consumption patterns are easily identified. By contrast, the searched network for the dish washer integrates more convolutional layers to capture continuously varying power consumption characteristics. For the microwave, the personalized model contains several pooling layers to focus on localized information of short duration.

TABLE I
COMPARISON OF MODEL PERFORMANCE ON THE REFIT DATASET

Appliance	Metric	Centralized	Local	NAS	MNAS	Federated	Proposed
Fridge	MAE	32.15	30.14	28.41	28.5	29.05	27.99
	SAE	1.131	1.125	1.042	1.057	1.083	1.034
Washing machine	MAE	18.25	23.7	19.55	19.05	21.94	17.34
	SAE	1.508	1.781	1.742	1.525	1.821	1.460
Dish washer	MAE	38.42	39.36	36.94	36.99	36.58	36.11
	SAE	1.176	1.223	1.167	1.173	1.14	1.133
Microwave	MAE	9.52	10.22	9.24	9.49	9.56	8.58
	SAE	1.496	1.797	1.498	1.521	1.485	1.401
Kettle	MAE	20.64	23.25	21.03	20.81	20.75	18.24
	SAE	1.813	2.012	1.796	1.789	1.856	1.552

TABLE II
COMPARISON OF MODEL PERFORMANCE ON THE REDD DATASET

Appliance	Metric	Centralized	Local	NAS	MNAS	Federated	Proposed
Fridge	MAE	32.78	34.45	32.83	32.47	32.99	31.73
	SAE	0.475	0.496	0.478	0.471	0.476	0.462
Dish washer	MAE	11.74	9.63	9.06	8.84	8.8	8.51
	SAE	1.265	1.121	1.107	1.097	1.052	1.008
Microwave	MAE	18.74	20.52	19.22	18.96	18.31	17.89
	SAE	1.126	1.22	1.158	1.125	1.098	1.081

We compare the performance of our proposed model with benchmark models using the REFIT and REDD datasets. The performance of the REFIT and REDD are summarized in Table I and Table II respectively with the best model highlighted in bold. In both two datasets, our proposed model achieves the best performance with MAE and SAE for all appliances. Using the local model as the baseline case, we can observe that the centralized model outperforms the local model for most appliances by acquiring data from other households. Both NAS and MNAS models outperform the local model by introducing architecture search before the model training stages. Notably, MNAS yields comparable and even higher accuracy to NAS on the REDD dataset. The possible reasons include: 1) the efficiency of the compressed search space enhances model generalization, 2) the focusedness of the single-path search strategy accelerates model convergence, and 3) the regularization of the hardware-aware evaluation mitigates model overfitting. A detailed comparison of the model efficiency will be carried out in later sections. The federated model without architecture search shows better performance compared to the local model across most appliances. This is expected since federated learning empowers the local model with knowledge from other households while preserving privacy. Overall, the local model shows the lowest accuracy with only a few exceptions where appliance usage patterns are better captured using only local data. Our proposed model, uniting the strength of architecture search and federation, shows the best performance.

C. Model Effectiveness

1) *Performance on Different Appliances*: To clearly and intuitively demonstrate the enhancements brought by the proposed method in terms of energy assessment and monetary gain, we further incorporate the MAE in units of kWh as an additional metric in our experiment. We summarize the relative improvement of on-device models compared with the base local model over the REFIT and REDD datasets in Fig. 7. The

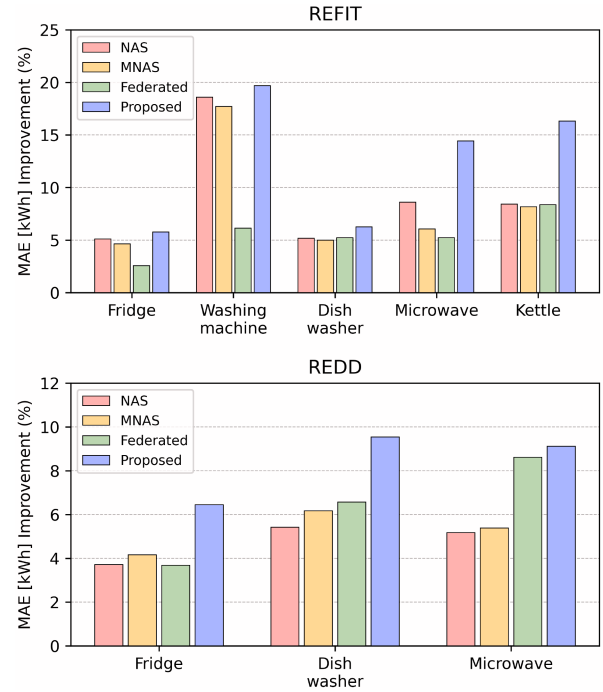


Fig. 7. Relative improvements in energy consumption estimation of different methods for different appliances.

centralized model is excluded since it can not be implemented on end devices. We can observe a larger improvement in the washing machine, microwave, and kettle with over 10% MAE enhancement for the proposed model. The improvement for the fridge is not significant, which may be due to the simplicity of its prediction task. Both the NAS and the MNAS models have a competitive edge over the federated model for the fridge and washing machine, indicating that local knowledge is effective in these appliances. For the dish washer, global knowledge contributes to better prediction accuracy. Our proposed model combines the MNAS and federated

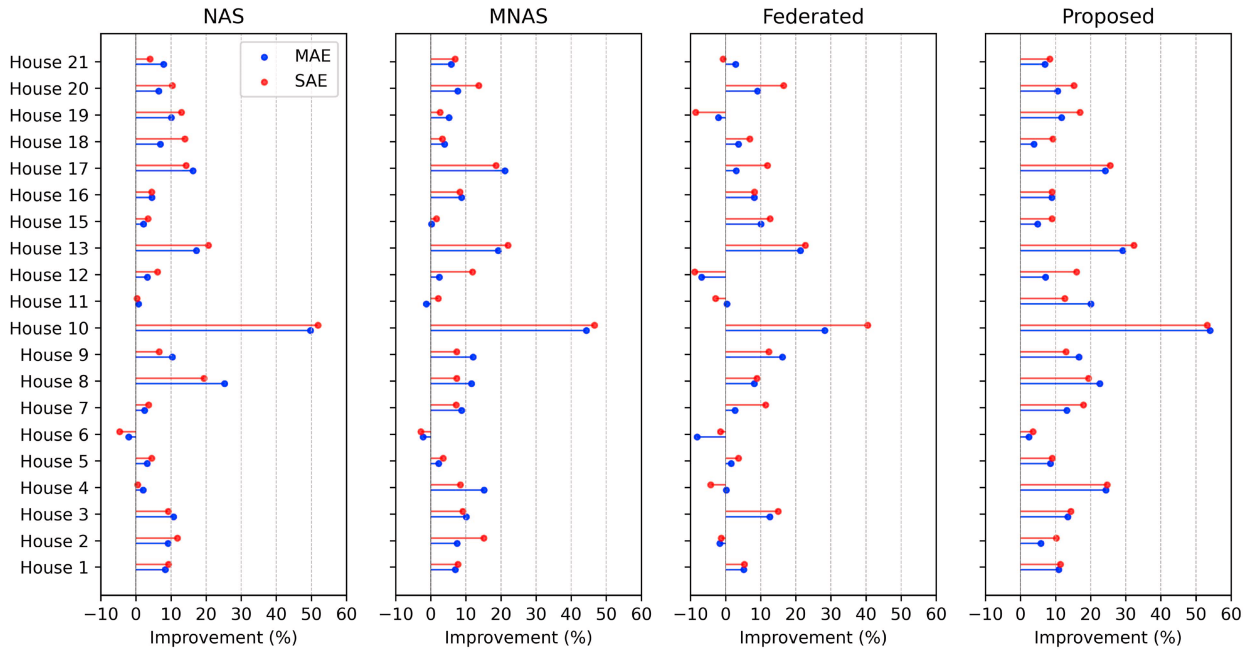


Fig. 8. Relative improvements of different methods for different houses on REFIT dataset.

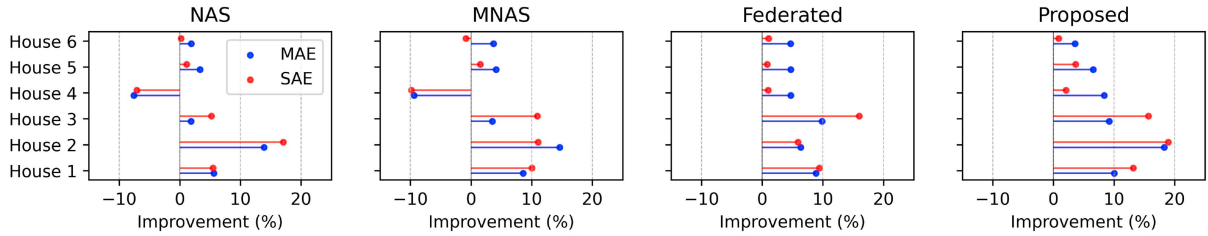


Fig. 9. Relative improvements of different methods for different houses on REDD dataset.

learning approaches, resulting in an overall improvement in energy consumption estimation. This highlights the benefits of harnessing both local and global knowledge.

2) *Performance on Different Households*: Fig. 8 shows the relative improvement for the baseline of different households in the REFIT dataset. In general, all houses benefit from the proposed method with universal performance improvement. However, the other on-device models show a minor degradation on some specific houses, such as House 6. This is because House 6 owns 49 appliances, which is the most among other households. Excessive appliance loads complicate the disaggregation task by bringing more uncertainty. In contrast, by integrating local and global knowledge in our model, the aforementioned underperforming House 6 can have a better performance than the baseline model. In addition, Houses 2, 4, and 19 show an improving accuracy using NAS and MNAS and degrading performance using federated learning. Such households have distinctive consumption patterns and, hence, do not benefit from the inclusion of global knowledge. The accuracy improvement is more pronounced for houses 4 and 17 using MNAS compared to NAS, which may be attributed to the fact that the localized information better reflects their frequent total power variations, whereas MNAS focuses more on searching with smaller kernel-size convolutional layers in the

compressed supernet. Fig. 9 shows the household-wise relative improvement for the REDD dataset. Our proposed model outperforms other benchmarks by showing improvement across all 6 households. Federated learning also shows a universal improvement, while NAS and MNAS show degraded accuracy when applied to House 4. This phenomenon can be explained by the fact that the REDD dataset contains a shorter period compared to the REFIT dataset, and therefore the inclusion of global information improves the model performance by solving the local data paucity problem.

D. Model Robustness

1) *Data Imbalance*: Large-scale end devices may exhibit varying amounts of data due to instances of device failure, device replacement, and data loss. This imbalance may drive the global model to favor devices with larger data volumes. We conducted experiments to explore the influence of data imbalance on the performance of our proposed method. The imbalance is generated by transferring 30% of the original data from a portion of the devices to the remaining devices. Table III demonstrates the MAE results with the imbalance ratio on the REFIT dataset. We can observe a certain growth in decomposition error as the imbalance ratio increases due

TABLE III
MODEL PERFORMANCE WITH DIFFERENT IMBALANCE RATIO

Appliance	Imbalance ratio			
	0%	10%	25%	50%
Fridge	27.99	28.13	28.35	29.59
Washing machine	17.34	18.05	18.86	20.23
Disher washer	36.11	36.49	36.63	37.78
Microwave	8.58	8.89	9.01	9.59
Kettle	18.24	18.97	19.41	20.44
Average	21.65	22.10	22.45	23.52

TABLE IV
MODEL PERFORMANCE WITH DIFFERENT HETEROGENEITY RATIO

Appliance	Heterogeneity ratio			
	0%	10%	25%	50%
Fridge	27.99	28.05	28.14	28.40
Washing machine	17.34	17.36	17.61	18.19
Disher washer	36.11	36.05	36.73	37.38
Microwave	8.58	8.67	8.83	9.05
Kettle	18.24	18.40	18.79	19.11
Average	21.65	21.70	22.02	22.43

to the high contribution of data volume to model generalization. Overall, the proposed method maintains satisfactory performance with no more than a 4% accuracy drop, except in extreme cases where half of the devices are affected by data imbalance issues.

2) *Device Heterogeneity*: Large-scale end devices may encounter varying local training durations due to differences in real-time task occupancy, network communication velocity, and operating frequency configuration. This heterogeneity could potentially cause asynchronous uploading of model weights to the central server. We carried out experiments to investigate the effect of device heterogeneity on the performance of our proposed method. The heterogeneity is introduced by prolonging the training time of a portion of the devices to twice the original duration. Table IV presents the MAE results with the heterogeneity ratio on the REFIT dataset. Remarkably, asynchronous aggregation with insignificant heterogeneity can almost match or even surpass the accuracy of training in a non-heterogeneity environment for certain appliances. Even though half of the devices experience delays, the proposed method still achieves stable accuracy, demonstrating significant robustness against device heterogeneity.

E. Complexity Analysis

To compare the model efficiency of NAS and the proposed method, we provide an analysis in terms of time complexity and space complexity. Let $\mathcal{E}$ and $\mathcal{O}$ denote the set of edges and candidate operations for each path, respectively. Additionally, let $K_{i,j}$, $M_{i,j}$ and C_j denote the kernel size, feature map length, and channel number of i -th operation on the j -th edge, respectively. For searching architecture with NAS, the time complexity can be expressed as $Time \sim O(\sum_{j \in \mathcal{E}} \sum_{i \in \mathcal{O}} K_{i,j} \cdot M_{i,j} \cdot C_{j-1} \cdot C_j)$ and the space complexity (i.e., memory footprint) can be expressed as $Space \sim O(\sum_{j \in \mathcal{E}} \sum_{i \in \mathcal{O}} K_{i,j} \cdot C_{j-1} \cdot C_j + M_{i,j} \cdot C_j)$. In our approach, both the search space and the search strategy are improved to increase model efficiency. Specifically, only the largest convolution kernel needs to be

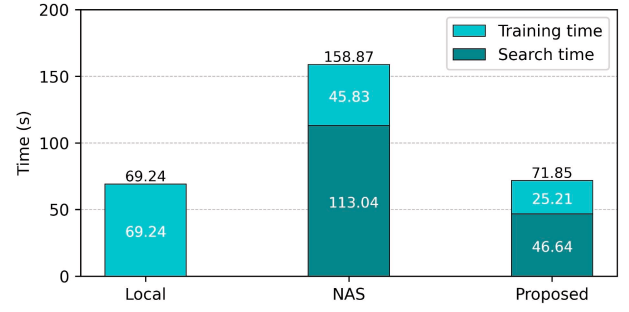


Fig. 10. Comparison of training efficiency for different methods.

TABLE V
COMPARISON OF TESTING EFFICIENCY FOR DIFFERENT METHODS

Method	Model size (KB)	Inference time (s)
Local	29.056	6.55E-03
NAS	10.979	4.18E-03
Proposed	7.011	2.53E-03

stored, and only one path needs to be activated in memory per search step. Consequently, the time complexity can be expressed as $Time \sim O(\sum_{j \in \mathcal{E}} K_j^* \cdot M_j^* \cdot C_{j-1} \cdot C_j)$ and the space complexity can be expressed as $Space \sim O(\sum_{j \in \mathcal{E}} K_j^* \cdot C_{j-1} \cdot C_j + M_j^* \cdot C_j)$. Here K_j^* and M_j^* denote the maximum kernel size and feature map length of j -th edge. Thus the proposed method can save roughly M times memory consumption on the end device compared to the NAS.

To evaluate the computational expense, we compare the training and testing efficiency of the non-search local model, the NAS model, and our proposed model. The training time and search time of three models for the fridge in the REDD dataset are shown in Fig. 10. It can be found that the proposed model remarkably reduces the search time and training time of NAS, bringing it to the same level as the local model. The model size and inference time for each sample are listed in Table V. The results show that the size of the customized architecture, i.e., the memory space required, is smaller than that of the fixed model. Prior research has examined the correlation between the memory required for model training and the model size, revealing an empirical disparity of approximately 20 times [39]. As such, the memory overhead necessary for compact model training can be estimated as 140.22 KB. This overhead is feasible for model deployment on widely used end devices, such as smart meters equipped with the STM32F405 microcontroller, which has a static random-access memory of 192KB. Moreover, the proposed hardware-aware method can limit disaggregating time to a preset sliding time window and reduce the latency time by more than 2.5 times. Note that the actual runtime on the end device will be longer than the above given time measured by the high-performance GPU device. Overall, the improved efficiency showcases the feasibility and effectiveness of our model for implementation on resource-constrained end devices.

V. CONCLUSION AND FUTURE WORKS

In this paper, we propose a lightweight federated learning approach to enable NILM on resource-constrained devices.

The developed memory-efficient NAS algorithm reduces computation resource consumption to the same level as compact model training. The proposed method develops personalized models with unique network architecture customized to various appliances with distinct usage patterns, boosting decomposition accuracy. Furthermore, the federated framework integrated with adaptive mutual learning can utilize distributed data to collaboratively train heterogeneous models in a privacy-preserving manner. The global knowledge transferred from unified proxy models further enhances the generalization of personalized models, with better performance than benchmark models. The results show significant improvements over 15% for appliances such as washing machines, microwaves, and kettles on the REFIT dataset, and an overall improvement of around 10% for different appliances on the REDD dataset. Importantly, our proposed method achieves considerable improvements in memory footprint and computational time. Furthermore, the proposed method demonstrates robustness in the presence of imbalanced data and device heterogeneity. In brief, our method offers a promising and feasible solution for on-device NILM applications.

However, our work also presents the following limitations. First, the improved NAS still slightly increases the computational cost on end devices since they need to learn customized model architecture from the search space. Second, our study presumes that the communication network and the aggregation server are both reliable and secure, thereby overlooking potential threats posed by hackers and malicious attackers.

As illustrated in the case studies, even though lightweight federated learning outperforms the benchmarks overall, federated learning and NAS achieve the best performance on specific appliances and households. An intriguing area of study would be to analyze whether local and global knowledge is gainful on a given task. In addition, communication overhead is also a critical efficiency metric for end devices. Thus, another direction for future work will focus on combining the proposed approach with gradient quantization techniques to alleviate communication costs in practice.

REFERENCES

- [1] Z. Liu, Z. Deng, S. Davis, and P. Ciaia, "Monitoring global carbon emissions in 2022," *Nature Rev. Earth Environ.*, vol. 4, pp. 205–206, Mar. 2023.
- [2] P. A. Schirmer and I. Mporas, "Non-intrusive load monitoring: A review," *IEEE Trans. Smart Grid*, vol. 14, no. 1, pp. 769–784, Jan. 2023.
- [3] H. Çimen, N. Çetinkaya, J. C. Vasquez, and J. M. Guerrero, "A microgrid energy management system based on non-intrusive load monitoring via multitask learning," *IEEE Trans. Smart Grid*, vol. 12, no. 2, pp. 977–987, Mar. 2021.
- [4] B. Liu, W. Luan, J. Yang, and Y. Yu, "The balanced window-based load event optimal matching for NILM," *IEEE Trans. Smart Grid*, vol. 13, no. 6, pp. 4690–4703, Nov. 2022.
- [5] W. Luan, R. Zhang, B. Liu, B. Zhao, and Y. Yu, "Leveraging sequence-to-sequence learning for online non-intrusive load monitoring in edge device," *Int. J. Elect. Power Energy Syst.*, vol. 148, Jun. 2023, Art. no. 108910.
- [6] T.-T.-H. Le, S. Heo, and H. Kim, "Toward load identification based on the Hilbert transform and sequence to sequence long short-term memory," *IEEE Trans. Smart Grid*, vol. 12, no. 4, pp. 3252–3264, Jul. 2021.
- [7] C. Zhang, M. Zhong, Z. Wang, N. Goddard, and C. Sutton, "Sequence-to-point learning with neural networks for non-intrusive load monitoring," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2604–2611.
- [8] P. A. Schirmer and I. Mporas, "Double fourier integral analysis based convolutional neural network regression for high-frequency energy disaggregation," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 3, pp. 439–449, Jun. 2022.
- [9] J. Chen, X. Wang, X. Zhang, and W. Zhang, "Temporal and spectral feature learning with two-stream convolutional neural networks for appliance recognition in NILM," *IEEE Trans. Smart Grid*, vol. 13, no. 1, pp. 762–772, Jan. 2022.
- [10] Y. Liu, J. Qiu, and J. Ma, "SAMNet: Toward latency-free non-intrusive load monitoring via multi-task deep learning," *IEEE Trans. Smart Grid*, vol. 13, no. 3, pp. 2412–2424, May 2022.
- [11] D. Garcia-Perez, D. Pérez-López, I. Díaz-Blanco, A. Gonzalez-Muniz, M. Dominguez-Gonzalez, and A. A. C. Vega, "Fully-convolutional denoising auto-encoders for NILM in large non-residential buildings," *IEEE Trans. Smart Grid*, vol. 12, no. 3, pp. 2722–2731, May 2021.
- [12] L. Wang, S. Mao, and R. M. Nelms, "Transformer for nonintrusive load monitoring: Complexity reduction and transferability," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18987–18997, Oct. 2022.
- [13] M. D'Incecco, S. Squartini, and M. Zhong, "Transfer learning for non-intrusive load monitoring," *IEEE Trans. Smart Grid*, vol. 11, no. 2, pp. 1419–1429, Mar. 2020.
- [14] Y. Han, K. Li, C. Wang, F. Si, and Q. Zhao, "Unknown appliances detection for non-intrusive load monitoring based on conditional generative adversarial networks," *IEEE Trans. Smart Grid*, vol. 14, no. 6, pp. 4553–4564, Nov. 2023.
- [15] S. Mari, G. Bucci, F. Ciancetta, E. Fiorucci, and A. Fioravanti, "An embedded deep learning NILM system: A year-long field study in real houses," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–15, Oct. 2023.
- [16] E. Tabanelli, D. Brunelli, A. Acquaviva, and L. Benini, "Trimming feature extraction and inference for MCU-based edge NILM: A systematic approach," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 943–952, Feb. 2021.
- [17] Y. Liu, Q. Xu, Y. Yang, and W. Zhang, "Detection of electric bicycle indoor charging for electrical safety: A NILM approach," *IEEE Trans. Smart Grid*, vol. 14, no. 5, pp. 3862–3875, Sep. 2023.
- [18] S. Ahmed and M. Bons, "Edge computed NILM: A phone-based implementation using MobileNet compressed by TensorFlow lite," in *Proc. 5th Int. Workshop Non-Intrusive Load Monit.*, 2020, pp. 44–48.
- [19] S. Sykiotis et al., "Performance-aware NILM model optimization for edge deployment," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 3, pp. 1434–1446, Sep. 2023.
- [20] Q. Luo, T. Yu, C. Lan, Y. Huang, Z. Wang, and Z. Pan, "A generalizable method for practical non-intrusive load monitoring via metric-based meta-learning," *IEEE Trans. Smart Grid*, vol. 15, no. 1, pp. 1103–1115, Jan. 2024.
- [21] S. Athanasoulas, S. Sykiotis, M. Kaselimi, A. Doulamis, N. Doulamis, and N. Ipiotis, "OPT-NILM: An iterative prior-to-full-training pruning approach for cost-effective user side energy disaggregation," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4435–4446, Feb. 2024.
- [22] Y. Zhang et al., "FedNILM: Applying federated learning to NILM applications at the edge," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 857–868, Jun. 2023.
- [23] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," 2017, *arXiv:1611.01578*.
- [24] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.
- [25] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," 2018, *arXiv:1806.09055*.
- [26] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," 2018, *arXiv:1812.00332*.
- [27] "General data protection regulation." 2018. [Online]. Available: <https://gdpr-info.eu/>
- [28] S. Lee and D.-H. Choi, "Federated reinforcement learning for energy management of multiple smart homes with distributed energy resources," *IEEE Trans. Ind. Informat.*, vol. 18, no. 1, pp. 488–497, Jan. 2022.
- [29] H. Wang, C. Si, G. Liu, J. Zhao, F. Wen, and Y. Xue, "Fed-NILM: A federated learning-based non-intrusive load monitoring method for privacy-protection," *Energy Convers. Econ.*, vol. 3, no. 2, pp. 51–60, 2022.
- [30] T. Wang and Z. Dong, "Blockchain-based clustered federated learning for non-intrusive load monitoring," *IEEE Trans. Smart Grid*, vol. 15, no. 2, pp. 2348–2361, Mar. 2024.

- [31] S. Dai, F. Meng, Q. Wang, and X. Chen, "DP2-NILM: A distributed and privacy-preserving framework for non-intrusive load monitoring," *Renew. Sustain. Energy Rev.*, vol. 191, Mar. 2024, Art. no. 114091.
- [32] Q. Li, J. Ye, W. Song, and Z. Tse, "Energy disaggregation with federated and transfer learning," in *Proc. IEEE 7th World Forum Internet Things (WF-IoT)*, 2021, pp. 698–703.
- [33] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [34] F. Cincetta, G. Bucci, E. Fiorucci, S. Mari, and A. Fioravanti, "A new convolutional neural network-based system for NILM applications," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, Nov. 2020.
- [35] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," 2016, *arXiv:1611.01144*.
- [36] D. Murray, L. Stankovic, and V. Stankovic, "REFIT: Electrical load measurements (cleaned)," Dataset, Univ. Strathclyde, PURE, Glasgow, U.K., 2016.
- [37] J. Z. Kolter and M. J. Johnson, "REDD: A public data set for energy disaggregation research," in *Proc. Workshop Data Min. Appl. Sustain. (SIGKDD)*, San Diego, CA, USA, 2011, pp. 59–62.
- [38] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," 2019, *arXiv:1907.02189*.
- [39] N. S. Sohoni, C. R. Aberger, M. Leszczynski, J. Zhang, and C. Ré, "Low-memory neural network training: A technical report," 2019, *arXiv:1904.10631*.



Yehui Li (Graduate Student Member, IEEE) received the B.S. degree in electronic science and technology from the Harbin Institute of Technology in 2022. He is currently pursuing the Ph.D. degree in electrical and electronic engineering with The University of Hong Kong. His current research interests include data analytics and edge intelligence in smart grids.



Ruiyang Yao (Graduate Student Member, IEEE) received the M.Math. degree in mathematics and statistics from the University of Oxford and the M.S. degree in computing from Imperial College London. He is currently pursuing the Ph.D. degree in electrical and electronic engineering with The University of Hong Kong. His current research interests include data analytics and data security in smart grids.



Dalin Qin (Graduate Student Member, IEEE) received the B.S. degree in electrical engineering and its automation from the South China University of Technology, Guangzhou, China, in 2022. He is currently pursuing the Ph.D. degree in electrical and electronic engineering with The University of Hong Kong. His current research interests include energy forecasting and privacy-preserving data analytics in smart grids.



Yi Wang (Senior Member, IEEE) received the B.S. degree from the Huazhong University of Science and Technology in June 2014, and the Ph.D. degree from Tsinghua University in January 2019.

He was a visiting student with the University of Washington from March 2017 to April 2018. He served as a Postdoctoral Researcher with the Power Systems Laboratory, ETH Zurich from February 2019 to August 2021. He is currently an Assistant Professor with the Department of Electrical and Electronic Engineering, The University of Hong Kong. His research interests include data analytics in smart grids, energy forecasting, multienergy systems, Internet-of-Things, and cyber-physical-social energy systems.